How tree tensor networks learn: a plain-language summary

Short version of the full report (`report/main.pdf`)

October 2026

How to read this. Each result carries two labels. *Novelty*: NEW means we found no earlier statement of it; KNOWN IDEA, NEW FOR TREES means the mechanism was known in other models and we worked out what it does in tree networks; KNOWN means it was already known. *Evidence*: [proven] (a mathematical proof), [derived] (a calculation that relies on stated approximations), [empirical] (measured in simulations), [conjecture] (a hypothesis we have not tested). Every simulation claim was re-checked by an independent agent, except one, which is marked.

1 The setting in plain terms

The model. The input is N vectors $x_0, \dots, x_{N-1} \in \mathbb{R}^d$, for example N pixels. The model's output is a weighted sum over all products of one entry from each vector:

$$f(x) = \sum_{i_0,\dots,i_{N-1}} W_{i_0 i_1 \cdots i_{N-1}}\, x_0[i_0]\, x_1[i_1] \cdots x_{N-1}[i_{N-1}]. \tag{1}$$

The weight array W has d^N entries, which is too many to store, so a **tree tensor network** (TTN) builds W from K small arrays called **cores**. The cores sit at the nodes of a tree. Each input position (a **site**) attaches to one core, and neighbouring cores are joined by a shared index of size χ, the **bond dimension**. Summing over the shared indices gives W. A larger χ lets the network represent more tensors. We compare three tree shapes (Fig. 1):

- **MPS** (a chain): one core per site, $K = N$.
- **Binary tree**: sites attach in pairs to the bottom cores, with $K = N - 1$.
- **Hierarchical Tucker (HT)**: every site has its own leaf core, with a binary tree above, $K = 2N - 1$. The upper cores carry no sites.

The task. A *teacher* tensor W^* is a sum of R **components** with weights $s_1 > s_2 > \dots$:

$$W^* = \sum_{r=1}^{R} s_r\, u_0^{(r)} \otimes u_1^{(r)} \otimes \cdots \otimes u_{N-1}^{(r)}. \tag{2}$$

In our main examples each $u_n^{(r)}$ is a basis vector. A component can then be written as a string such as 00011000, which lists the basis vector used at each site. With whitened Gaussian inputs, the expected squared error is, up to a constant,

$$L = \tfrac{1}{2}\, \|W - W^*\|^2. \tag{3}$$

Training. We use **gradient flow**, $\dot{\theta} = -\nabla L(\theta)$ (gradient descent with very small steps), where θ collects all core entries. The cores start random with overall size σ, the **initialization scale**. The output f is a product of K cores, so it starts at a size of roughly σ^K.

Vocabulary for the dynamics.

- **Plateau**: a long stretch in which the loss barely moves. **Saddle-to-saddle** training means the loss falls in steps, with one plateau before each component is learned.
- **Saddle**: a point where the gradient is zero but which is not a minimum. Plateaus happen near saddles.
- **Order of a saddle**: the smallest p such that moving a distance ε in some direction lowers the loss by about ε^p. Order $p = 2$ is a **strict** saddle, where some direction curves downward. For $p \geq 3$ the loss is flat to second order, so escape is much slower.
- **Lazy regime**: the weights barely move during training, and the network behaves like a linear model with fixed features (a "kernel"). **Rich regime**: the features change, but without clear steps.

The one equation behind most results. Suppose m cores must all grow from size $\sim \sigma$ for the loss to fall, while the other cores are already of size $O(1)$. The gradient on each small core is a product of the other $m - 1$ small cores, so their common size a obeys

$$\dot{a} \approx c\, a^{m-1}, \qquad a(0) \sim \sigma \quad \Longrightarrow \quad T_{\text{plateau}} \approx \begin{cases} O(1) & m = 1, \\ \dfrac{\ln(1/\sigma)}{c} & m = 2, \\ \dfrac{\sigma^{-(m-2)}}{(m-2)\, c} & m \geq 3. \end{cases} \tag{4}$$

For $m \geq 3$ the solution blows up at a finite time, so the size stays near σ for a long time and then shoots up: this is the plateau and the sudden drop. Here c is proportional to the weight of the component being learned. Equation (4) is standard for products of weights. Our results concern *what* m *is* in a tree network, *which* cores are small, and when the simple picture fails.

2 Results at a glance

	Result	Novelty
R1	Learning is saddle-to-saddle; first plateau $\propto \sigma^{-(K-2)}$ with a prefactor predictable from the start	KNOWN IDEA, NEW FOR TREES
R2	**Route rule**: later plateaus are set by how many cores a new component needs, not by its weight	NEW (built on a known mechanism)
R3	**Saddle-order theorem**: when the route rule is exact, and when a faster shortcut exists	NEW
R4	The tree shape decides whether saddles are strict (MPS) or not (binary, HT)	NEW
R5	Which component is learned first: a random race or "largest first"	KNOWN IDEA, NEW FOR TREES
R6	Dead routes: a quantity fixed at initialization predicts which runs get stuck	NEW (related ideas known)
R7	Map of regimes: the lazy regime exists only with full bond dimension	NEW
R8	Optimizers: GD step-size limit, SGD, Adam, ALS	mixed, see text
R9	MNIST: the tree's first learned features, and what predicts the final loss	NEW (mechanism a conjecture)

3 The results explained

R1. One component at a time — KNOWN IDEA, NEW FOR TREES

From small σ the loss falls in steps, and each step learns one component [empirical]. Before the first step all K cores are small, so $m = K$ in (4):

$$T_1 \approx \frac{\hat{t}(Z)}{s_1\, \sigma^{K-2}}, \tag{5}$$

where Z is the random initial direction. We compute the number $\hat{t}(Z)$ by solving one σ-independent equation *before training starts*. It predicts the first escape time, and which component is learned first in 1,847 of 1,848 runs [derived, empirical]. The textbook estimate based only on the initial norm underestimates T_1, by up to a factor of $3 \cdot 10^7$. *Known:* the $\sigma^{-(K-2)}$ law and the per-initialization formula hold for any model that is a product of K weight layers (Kumar and Haupt, 2025; Kunin et al., 2025). *New:* evaluating them for trees and testing them run by run.

R2. The route rule: "locality beats weight" — NEW

After the first component is learned, most cores are already large. To learn the next component, the network needs to change only some of them:

- the cores holding the sites where the new component *differs* from what is already learned;
- every core on the tree path connecting those cores, because the new information has to pass through them.

We call this connected set of cores the **route**, and write k for its number of cores (in graph theory it is the Steiner tree of those cores). Only the route cores start small in the new direction, so $m = k$ in

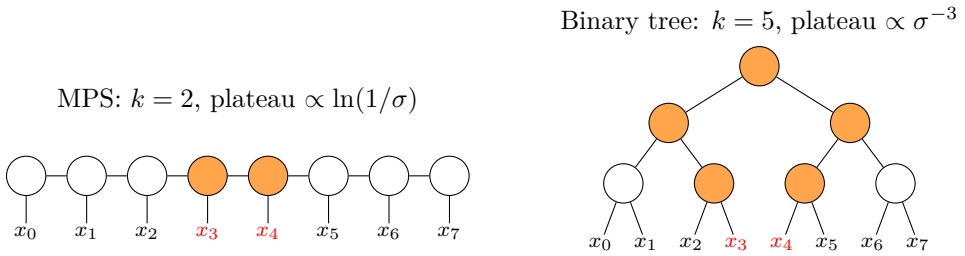


Figure 1: The route for the same task on two trees. The network has learned 00000000 and must now learn 00011000, which differs at sites x_3, x_4 (red). The **route** is the set of cores that must change (orange): the cores holding the differing sites, plus every core on the tree path between them. On an HT tree the route has $k = 7$ cores and the plateau grows like σ^{-5}.

(4):

$$T_{\text{next}} \propto \begin{cases} O(1) & k = 1 \text{ (no plateau)}, \\ \ln(1/\sigma) & k = 2, \\ \sigma^{-(k-2)} & k \geq 3. \end{cases} \tag{6}$$

We measured the exponent $k - 2$ for $k = 1, \ldots, 5$ in about 11,600 runs on nine trees [empirical]. The logic leading to it is [derived], and the count of route cores is [proven]. Two consequences:

- **Locality beats weight.** As $\sigma \to 0$, a *smaller* component with a shorter route is learned before a larger component with a longer one. This is the reverse of the usual "largest first" rule.
- **The same task gives different laws on different trees** (Fig. 1): logarithmic on an MPS, σ^{-3} on a binary tree, about σ^{-5} on HT [empirical].

Known: Rawal and DeWeese (2026) give escape laws in terms of the number of small layers, in an abstract form. *New:* identifying which cores are small (the route) and how this depends on the tree.

R3. The saddle-order theorem

NEW **[proven]**

R2 assumes that the saddle after a learned component has order $p = k$. We proved when this is exactly true [proven, checked twice independently]:

- **When it holds.** Let s_j be the weight of the learned component. We define a matrix $\mathcal{S}$ that measures how strongly the components still to be learned couple through small groups of cores. If its largest eigenvalue satisfies $\lambda_{\max}(\mathcal{S}) < s_j$, the order is exactly k. This always holds when the teacher has two components learned largest first, and also for routes of 3 or 4 cores under a mild condition.
- **When it fails.** If $\lambda_{\max}(\mathcal{S}) > s_j$, the network can take a shortcut: it splits the route into two pieces of at most ℓ cores each and escapes with order $2\ell < k$. This can happen when a larger component is still unlearned, or when several unlearned components share cores. The first counterexample has $k = 5$ and order 4, on an MPS with 6 sites.

What this predicts for training: where the shortcut exists, escape is faster than the route rule says. A pre-registered test (about 2,800 runs) measured exponents 2.001 (order 4, route rule 3) and 3.996–3.998 (order 6, route rule 5), as predicted, when the starting point is close enough to the saddle; results were mixed further away [empirical; **not** independently re-checked]. *Known:* an *upper* bound on the order (no larger than the number of rank-deficient cores, Furman et al., 2026); lower bounds only for deep

linear networks (Achour et al., 2024). *New:* that the relevant number is the route size, which can be much smaller, plus the exact condition and the shortcut.

R4. Tree shape decides whether saddles are strict

NEW **[derived, empirical]**

When the teacher's components are not exactly orthogonal, which is the generic case, the components overlap, and this overlap creates new routes of only two cores, but only between *neighbouring cores that both carry sites*. An MPS has such pairs everywhere, so its saddles become strict ($p = 2$) and the plateaus become logarithmic. In binary and HT trees the internal cores carry no sites, so the saddles stay higher-order and the plateaus stay power laws (13 teachers, $N = 4$ and 8).

R5. Which component comes first

KNOWN IDEA, NEW FOR TREES

Suppose two components compete, and they differ from each other at sites spread over k cores.

- $k \geq 3$**: a race.** By (4) each finishes at $T_r \propto (\sigma z_r)^{-(k-2)}/s_r$, where z_r is a random number from the initialization. The ratio T_1/T_2 does not depend on σ, so the winner is random with fixed odds [proven at leading order on MPS; empirical, 67,200 runs].
- $k = 2$**: greedy.** Now $T_r \approx \ln(1/\sigma)/s_r$, so the larger component wins with probability $\to 1$ as $\sigma \to 0$ [derived, empirical].

Known: greedy learning in deep linear networks (Saxe et al., 2014; Jacot et al., 2022), and races between spikes in tensor PCA (Ben Arous et al., 2025). *New:* the split by number of cores, and the exact race on MPS.

R6. Dead routes: getting stuck for good

NEW **(related ideas known)**

Some runs never learn a component, because a core at an *end* of its route shrinks to numerical zero (398 of 5,682 runs, all with $k \geq 4$; most runs with $K = 7, 8$ at tight χ) [empirical]. Gradient flow conserves, for every bond, the difference of the squared sizes of the two cores it joins (as in deep linear networks). For each route end, the sign of this conserved quantity, which is fixed at initialization, decides whether that end can grow or must die [derived]. The prediction from the initialization alone works: 93% of runs predicted dead stall, against 1.8% of runs predicted alive [empirical]. Initializing with balanced core sizes prevents leaf ends from dying. *Known:* stalls caused by unbalanced initialization (Furman et al., 2026). *New:* the route-end criterion and its probabilities.

R7. Map of regimes

NEW

As σ grows, training goes from saddle-to-saddle, through a rich window, to lazy [empirical]. The lazy regime exists *only if* every bond is full, meaning it has the largest rank it can possibly need:

$$\chi_e \;\geq\; \min\left(d^{n_e},\, d^{N-n_e}\right) \quad \text{for every bond } e, \tag{7}$$

where n_e is the number of sites below bond e. Without this, large-σ training stalls at a loss floor [derived, empirical]. On the binary tree with 4 sites, both boundaries scale as $\chi^{-1/2}$, with $\sigma_{\text{lazy}} \approx 6.5/\sqrt{\chi}$ [derived, empirical]. We also give a TTN version of the μP scaling, which keeps feature learning as χ grows [derived].

R8. Other optimizers — mixed

- **Gradient descent** with step η follows the same sequence of saddles if $\eta < \eta_c = 2/(K\, s_1^{2-2/K})$ [proven]; above η_c it oscillates or diverges. KNOWN IDEA, NEW FOR TREES (cf. MacDonald et al., 2025).
- **SGD** never shortens the first plateau on average. It lengthens later plateaus and can stall a run [empirical]. KNOWN IDEA, NEW FOR TREES
- **Adam** still learns one component at a time, but the first escape takes a number of steps independent of σ, until σ is so small that Adam's ϵ dominates [derived, empirical]. KNOWN (concurrent work, Sabanayagam and Lucey, 2026).
- **ALS, DMRG, Riemannian GD** (standard tensor-network optimizers) have no plateaus [derived].

R9. Real data: MNIST — NEW

On a binary tree over pixels, the first route learned is a pair of neighbouring cores ($k = 3$). A test designed before the data was seen compared 27 pixel orderings. A data-only measure of how strongly short routes couple predicts the final loss (rank correlation -0.60, 95% CI $[-0.82, -0.22]$) but not the training speed. Spatial closeness of pixels by itself is not the cause [empirical]. Why the measure works is a [conjecture].

4 What was already known

KNOWN The binary tree with 4 sites is mathematically a 3-layer linear network, so its stepwise learning, largest-first order and strict wrong-order saddles were known (Saxe et al., 2014; Jacot et al., 2022; Achour et al., 2024). Also known: the balanced escape direction from zero (Kumar and Haupt, 2025); the two-core case, which is matrix factorization (Jiang et al., 2023); the bias of hierarchical tensor factorization toward low rank (Razin et al., 2022). To our knowledge no earlier paper describes saddle-to-saddle dynamics of tree networks with three or more cores.

5 Main caveats

- Small trees ($N \leq 16$, mostly $N \leq 8$), exact expected loss, whitened Gaussian inputs, and mostly synthetic teachers.
- The boundary between saddle-to-saddle and rich training for large trees is open.
- The MNIST mechanism is a conjecture.
- The R3 dynamics test was not independently re-checked.

The full report has all numbers, error bars, and a record of claims withdrawn after review.

References

El Mehdi Achour, François Malgouyres, and Sébastien Gerchinovitz. The loss landscape of deep linear neural networks: a second-order analysis. *Journal of Machine Learning Research*, 25(242):1–76, 2024.

Gérard Ben Arous, Cédric Gerbelot, and Vanessa Piccolo. Stochastic gradient descent in high dimensions for multi-spiked tensor PCA. arXiv preprint arXiv:2410.18162, 2025.

Zach Furman, Stephan Wäldchen, Yangda Bei, and Liam Hodgkinson. Benign loss landscapes can coexist with worst-case hardness. arXiv preprint arXiv:2609.13057, 2026.

Arthur Jacot, François Ged, Berfin Şimşek, Clément Hongler, and Franck Gabriel. Saddle-to-saddle dynamics in deep linear networks: Small initialization training, symmetry, and sparsity. arXiv preprint arXiv:2106.15933, 2022.

Liwei Jiang, Yudong Chen, and Lijun Ding. Algorithmic regularization in model-free overparametrized asymmetric matrix factorization. *SIAM Journal on Mathematics of Data Science*, 5(3):723–744, 2023. doi: 10.1137/22M1519833.

Akshay Kumar and Jarvis Haupt. Early directional convergence in deep homogeneous neural networks for small initializations. *Transactions on Machine Learning Research*, 2025.

Daniel Kunin, Giovanni Luca Marchetti, Feng Chen, Dhruva Karkada, James B. Simon, Michael R. DeWeese, Surya Ganguli, and Nina Miolane. Alternating gradient flows: A theory of feature learning in two-layer neural networks. In *Advances in Neural Information Processing Systems*, volume 38, pages 4951–4998, 2025. doi: 10.52202/085713-0156.

Lachlan Ewen MacDonald, Hancheng Min, Leandro Palma, Salma Tarmoun, Ziqing Xu, and René Vidal. Convergence rates for gradient descent on the edge of stability for overparametrised least squares. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Divit Rawal and Michael R. DeWeese. A theory of saddle escape in deep nonlinear networks. arXiv preprint arXiv:2605.01288, 2026.

Noam Razin, Asaf Maman, and Nadav Cohen. Implicit regularization in hierarchical tensor factorization and deep convolutional neural networks. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18422–18462, 2022.

Mahalakshmi Sabanayagam and Simon Lucey. Optimizer geometry sets the pace: Spectral learning dynamics in matrix factorization. arXiv preprint arXiv:2610.04249, 2026.

Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations*, 2014.

A Figures (from the full report)

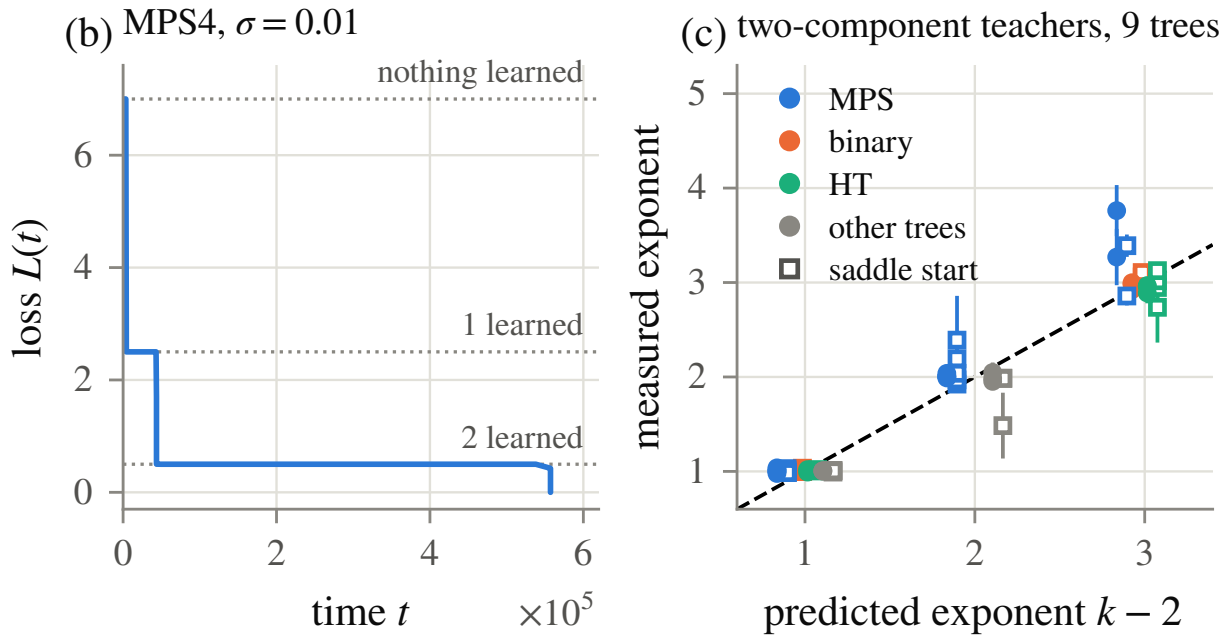


Figure 2: Saddle-to-saddle learning. Left: the loss falls in steps, with plateaus at the predicted partial-fit levels. Right: the measured plateau exponent against the route rule's $k - 2$ (full report, Fig. 1).

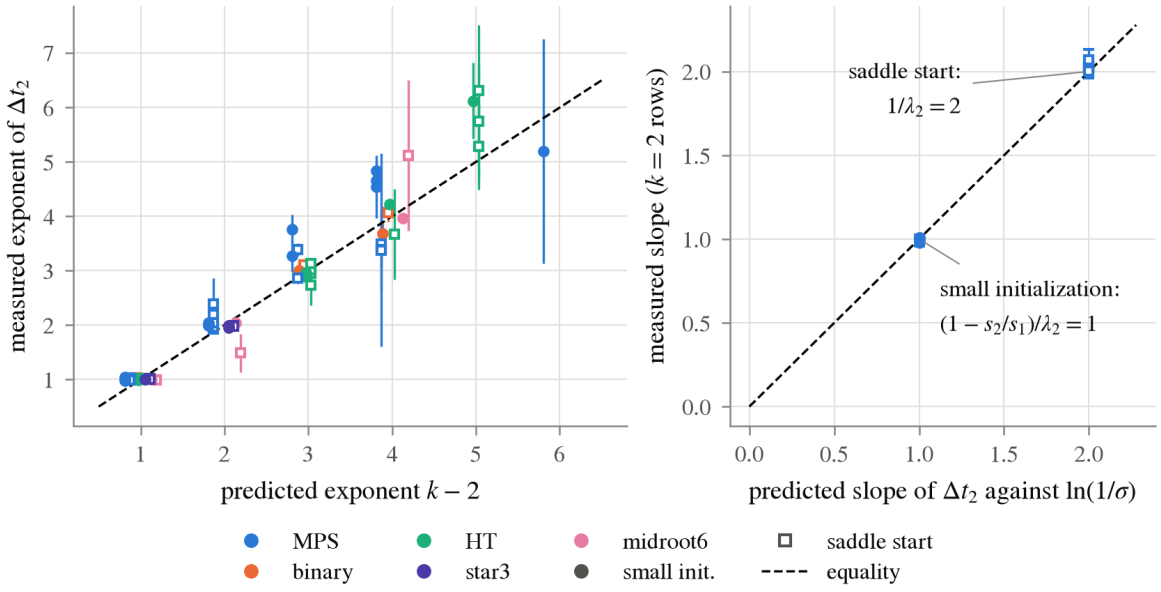


Figure 3: Plateau exponent against route size k across nine trees. The measurements follow $k-2$ (R2; full report, Sec. 4.4).

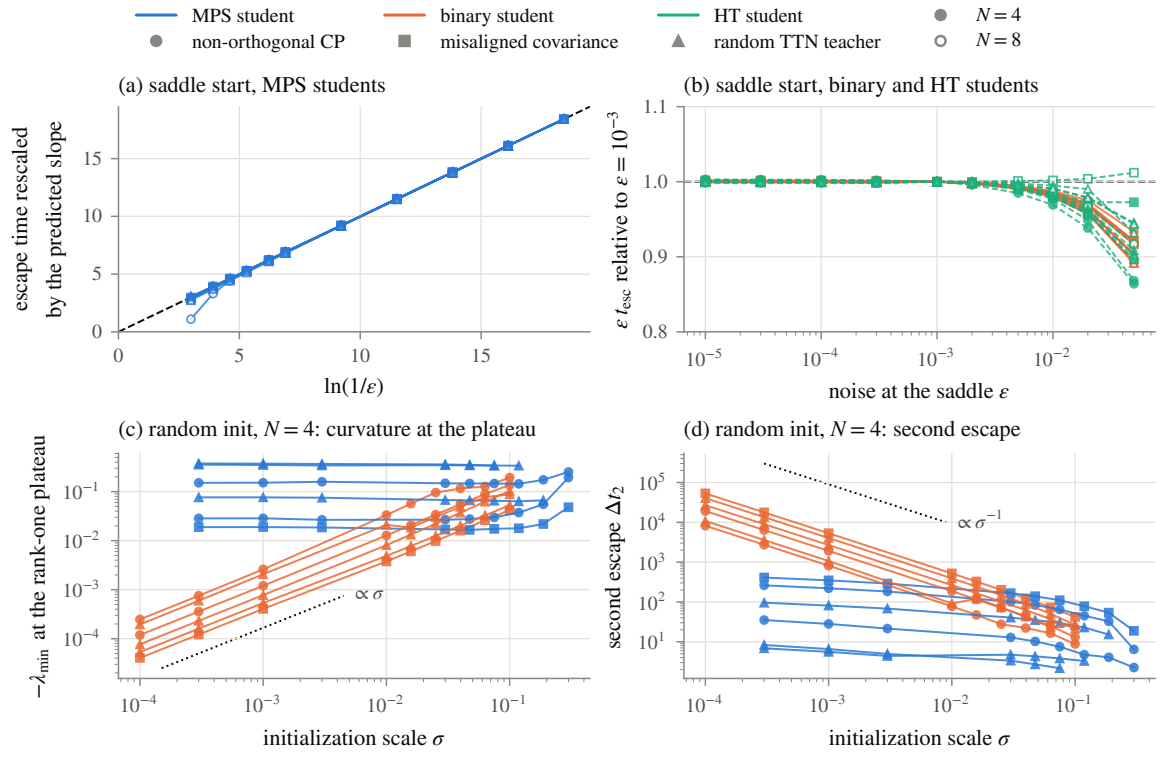


Figure 4: Tree shape and generic teachers (R4): logarithmic, strict escape on an MPS; power-law, higher-order escape on binary and HT trees (full report, Sec. 6.2).

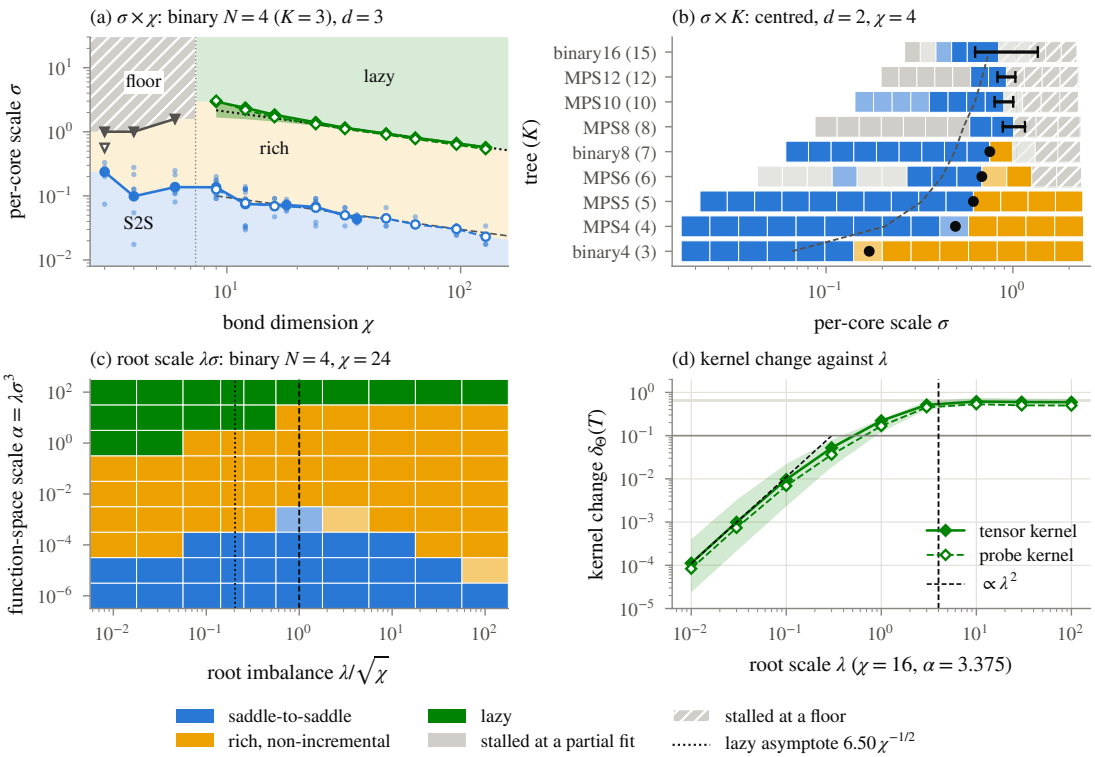


Figure 5: Map of regimes (R7) on the binary tree with 4 sites, by initialization scale σ and bond dimension χ (full report, Sec. 3).